# 1 Introduction to Transaction Management

A **transaction** is a logical unit of work that consists of one or more database operations. Transaction Management ensures that these operations are executed **reliably and consistently**.

A transaction must follow **ACID properties**:

- **A – Atomicity**: All operations succeed or all fail
- **C – Consistency**: Database remains in a valid state
- **I – Isolation**: Transactions do not interfere with each other
- **D – Durability**: Committed data is permanent

**ACID Properties – Real-Time Explanation**

**A – Atomicity (All or Nothing)**

**Definition:**
A transaction must either **complete fully** or **not execute at all**.

**Real-Time Example (ATM Transaction):**

- ₹10,000 is debited from Account A
- ₹10,000 is credited to Account B

If credit fails due to network issue:
Debit must be **rolled back**

Ensures no partial updates

---

**C – Consistency (Valid State)**

**Definition:**
A transaction must take the database from **one valid state to another**.

**Real-Time Example (Banking Rule):**

- Account balance cannot be negative
- Total money before = total money after transfer

If a rule is violated:
Transaction is **rolled back**

Business rules & constraints are always maintained

---

**I – Isolation (No Interference)**

**Definition:**
Multiple transactions executing at the same time **should not affect each other**.

**Real-Time Example (Online Ticket Booking):**

- Two users try to book the **same seat**
- Only one booking succeeds
- Other transaction waits or fails

Prevents dirty reads, lost updates

---

**D – Durability (Permanent Data)**

**Definition:**
Once a transaction is committed, data **will not be lost**, even after failures.

**Real-Time Example (Power Failure):**

- Money transfer is successful
- System crashes immediately after
- Data remains saved in database

Ensured using logs, backups, and disk storage

---

# 2 Local Transaction vs Distributed Transaction

## Local Transaction

A **local transaction** involves **only one resource**, usually a **single database**.

**Characteristics:**

- Uses a single DataSource
- Managed by JDBC or ORM (Hibernate)
- Simple and fast
- Limited to one database

**Example:**

```
Connection con = dataSource.getConnection();
con.setAutoCommit(false);
```

**Use Case:**

- One application
- One database

---

## Distributed Transaction

A **distributed transaction** involves **multiple resources**, such as:

- Multiple databases
- Database + JMS
- Multiple microservices

**Characteristics:**

- Managed using **XA protocol**
- Uses **Two-Phase Commit (2PC)**
- Complex and slower
- Requires transaction manager (JTA)

**Example:**

DB1 → Update
DB2 → Update
Message Queue → Send

**Use Case:**

- Enterprise applications
- Banking systems
- Microservices architecture

---

### Comparison Table

| Feature | Local Transaction | Distributed Transaction |
|---|---|---|
| **Resources** | Single | Multiple |
| **Complexity** | Low | High |
| **Performance** | Faster | Slower |
| **Manager** | JDBC / ORM | JTA / XA |
| **Rollback scope** | One DB | Multiple systems |

---

# 3 Need of Spring Transaction Management

Spring provides a **consistent, simple, and powerful** way to manage transactions.

### Problems without Spring

- Manual transaction handling
- Boilerplate code
- Tight coupling to APIs
- Error-prone rollback logic

Example (without Spring):

```
try {
    con.setAutoCommit(false);
    // business logic
    con.commit();
} catch (Exception e) {
    con.rollback();
}
```

---

## Advantages of Spring Transaction Management

### . Declarative Transaction Management

Use annotations instead of code:

```
@Transactional
public void transferMoney() {
   debit();
   credit();
}
```

---

### 2. Consistent Abstraction

Spring hides underlying APIs:

- JDBC
- Hibernate
- JPA
- JTA

You write **one style of code**.

---

### 3. Automatic Rollback

- Rolls back on runtime exceptions
- Configurable rollback rules

---

### 4. Supports Both Local & Distributed Transactions

- `DataSourceTransactionManager` → Local
- `JtaTransactionManager` → Distributed

---

### 5. Separation of Concerns

- Business logic is clean
- Transaction logic is externalized

---

**4.Conclusion**

Spring Transaction Management:

- Simplifies transaction handling
- Reduces boilerplate code
- Improves reliability
- Supports enterprise-level applications

**That’s why Spring Transaction Management is essential in real-world applications.**